



Python / C++

▼ Python

💡 기억해야 할 것들

- C++은 `{ }` 로 구분 / 파이썬은 들여쓰기로 구분.
- 위에서부터 한줄한줄 읽으면서 출력함.
- 변수 선언할 때 자료형을 쓰지 않음.

자세한 C++과 Python 구분 내용들은 맨 아래 확인

▼ if / for / while / break / continue 간단 예시

example 1

```
# example_1.py

sensor_values = [0, 0, 1, 0, 0]

for value in sensor_values:
    if value == 1:
        print("🚨 장애물 발견! 정지")
        break
    print("이동 중...")
```

! 조건문 내에는 "==" ("=" 아님)

example 2

```
# example_2.py

battery = 100

while battery > 0:    # False 될 때 까지 돌림
    print(f"현재 배터리: {battery}")

    if battery < 30:
        print("⚠ 충전소로 이동")
        break

    battery -= 15
```

example 3

```
# example_3.py
```

```

sensor_data = [25, 27, 999, 26, 999, 24] # 999 = 오류 값

for data in sensor_data:
    if data > 100: # 비정상적인 값이면
        print("△ 노이즈 감지: 건너뛴")
        continue

    print(f"정상 데이터 처리: {data}도")

```

▼ 리스트 / 튜플 / 딕셔너리

💡 우리가 이것들을 배우는 이유

ROS2 (센서)메시지 데이터 → 리스트/딕셔너리 형태

- 센서 데이터 → 배열(list)
- 상태 정보 → key-value(dict)

리스트 간단 예시

```

lidar_data = [0.5, 1.2, 0.3, 2.0]

for distance in lidar_data:
    if distance < 0.5:
        print("충돌 위험!")

```

튜플 간단 예시

```

position = (10, 20)

position[0] = 5 # ❌ 에러

```

리스트와 형태 비슷하지만, 수정할 수 없음.

주로 아래 상황에서 사용함.

- 좌표 (x, y)
- 고정 데이터
- 실수 방지용

딕셔너리 간단 예시 (1)

```

robot_status = {
    "battery": 25,
    "obstacle": True,
    "speed": 0.5
}

if robot_status["battery"] < 30:
    print("충전 필요")

```

```
if robot_status["obstacle"]:
    print("정지!")
```

딕셔너리 간단 예시 (2)

```
robots = [
    {"name": "R1", "battery": 90},
    {"name": "R2", "battery": 40},
    {"name": "R3", "battery": 20}
]

for robot in robots:
    if robot["battery"] < 30:
        print(f"{robot['name']} → 충전 필요")
    else:
        print(f"{robot['name']} → 정상")
```

▼ 함수

▼ C++

▼ 기본 형태 (test.cpp)

```
#include <헤더파일 이름>

int main(){

    (... 세부 코드 ...)

    return 0; // 프로그램 정상 종료
}
```

test.cpp 실행 방식

```
g++ test.cpp -o test && ./test // test라를 파일로 저장하고 test를 읽어오는 방식
```

예시 코드 (배터리 표시)

std::cout은 터미널 출력을 나타냄

```
#include <iostream>

int readBatterySensor() {
    // 실제로는 하드웨어 SDK나 ROS2 토픽 등을 통해 값을 가져옴.
    return 75;
}

int main() {
    int battery = readBatterySensor(); // 센서 데이터가 입력되는 지점
```

```
std::cout << "Sensor Data: " << battery << "%" << std::endl;
return 0;
}
```

```
g++ test.cpp -o test && ./test"
```

```
// [ 터미널 출력 형태 ]
Sensor Data: 75%
```

예시코드 (std::getline, std::cin - 입력)

```
#include <iostream> // input, output stream
#include <string>    // 문자열 사용하기 위함.

int main() {
    int ret = 1;
    // 0. 문자열 변수 선언 및 초기화 (보통 기본값은 "-"로 함. 빈칸 또는 공백은 인식 힘들.)
    std::string robot_description = "-";
    // 1. 타이틀 출력
    std::cout << "Enter Robot Desc: ";
    // 2. 설명 입력
    // std::cin >> robot_description; ← 원래 but 공백포함 저장하고 싶어 getline 사용.
    std::getline(std::cin, robot_description);
    // 3. 입력된 설명 출력
    std::cout << "Robot Desc: " << robot_description << std::endl;
    // 4. 반환값 변경
    ret = 0;
    // 5. 프로그램 종료
    return ret;
}
```

예시코드 (로봇 세부 내용 저장 코드)

```
#include <iostream>
#include <string>

int main() {
    // 1. 로봇 이름을 저장하는 std::string 변수 만들기
    std::string robot_name = "Warehouse_bot";

    // 2. 로봇 ID를 저장하는 int 변수 만들기
    int robot_id = 12345;

    // 3. 최대 속도를 저장하는 double 변수 만들기
    double max_speed = 25.0; // m/s

    // 4. 자율주행 가능 여부를 저장하는 bool 변수 만들기
    bool autonomous = true;

    // 5. 로봇 등급을 저장하는 char 변수 만들기
    char grade = 'A';

    // 6. bool 값은 true/false로 출력하기 (미리 선언한 것.)
```

```

std::cout << std::boolalpha;

    // 7. 모든 값을 보기 좋게 출력하기
std::cout << "==== Robot Information =====" << std::endl;
std::cout << "ROBOT NAME : " << robot_name << std::endl;
std::cout << "ROBOT ID : " << robot_id << std::endl;
std::cout << "Max Speed : " << max_speed << " m/s" << std::endl;
std::cout << "Autonomous : " << autonomous << std::endl;
std::cout << "Grade : " << grade << std::endl;

return 0
}

```

✓ 출력 결과

```

root@d1b87d26ec88:/workspace# g++ test2.cpp -o test3 && ./test3
==== Robot Information =====
ROBOT NAME : Warehouse_bot
ROBOT ID : 12345
Max Speed : 25 m/s
Autonomous : true
Grade : A

```

▼ for

🔄 실행 순서

```

for ( 1.초기화 ; 2.조건검사 ; 4.증감식 ) {
    3.본문 실행 (std::cout)
}

```

📄 예시코드 (i++, ++j 차이 확인 코드)

```

#include <iostream>
#include <string>

int main(){
    for(int i=0; i<5; ){
        std::cout << "i++ : [" << i++ << "]" << std::endl;
    }

    std::cout << "======" << std::endl;

    for(int j=0; j<5; ){
        std::cout << "++j : [" << ++j << "]" << std::endl;
    }
    return 0;
}

// i++ , ++i 가 있음. ++이 앞에 있으면 더하기 부터 하는 것.
// 4.증감식에 i++,++j 넣어도 똑같이 출력됨. 출력하고 증감하기 때문.

```

✓ 출력 결과

```

root@2e14ec01f30d:/workspace# g++ for_test.cpp -o test1 && ./test1
i++ : [0]
i++ : [1]
i++ : [2]
i++ : [3]
i++ : [4]
=====
++j : [1]
++j : [2]
++j : [3]
++j : [4]
++j : [5]

```

다른 형태의 for문

```

for (const auto& robot : robot_fleet) {
    // 내용물 사용
}

```

쉽게 이해가 안 되면, 아래 파이썬 예시를 보고 이해할 것.

```

for robot in robot_fleet:
    # 내용물 사용

```

▼ break

예시 코드

```

#include <iostream>

int main() {
    for (int step = 1; step <= 10; step++) {
        std::cout << "Step: " << step << std::endl;

        if (step == 5) {
            std::cout << "Emergency stop at step 5." << std::endl;
            break;
        }
    }

    return 0;
}

```

break 한 줄 요약 : "회전목마(Loop)에서 뛰어내리기"

- 탈출 대상: 나를 감싸고 있는 "가장 가까운 반복문(for, while) 또는 switch 문".
- if 는 "언제 뛰어내릴지" 결정하는 스위치일 뿐, break 가 탈출시키는 것은 결국 밖의 반복문.
- if 문은 어차피 한 번 하고 끝나므로 탈출할 벽이 X → 단독 if 에서 break 쓰면 컴파일 에러.

코드 한 줄 요약

```

for (...) {
    if (조건) break; // 👤 for문 루프 즉시 탈출 (OK)
}

// ❌ 잘못된 사용: 단독 if문
if (조건) break;    // ❌ 탈출할 루프 없음 (Error)

```

▼ continue

예시 코드

```
#include <iostream>

int main() {
    for (int sensor_id = 1; sensor_id <= 5; sensor_id++) {
        if (sensor_id == 3) {
            std::cout << "Sensor 3 skipped." << std::endl;
            continue;
        }

        std::cout << "Reading sensor " << sensor_id << std::endl;
    }

    return 0;
}
```

continue 한 줄 요약 : 이번 바퀴만 스킵 (다음 숫자로 점프).

sensor_id는 3일 때 continue로 빨려들어가고, 넘어가서 sensor_id++이 적용돼서 4가 되고 다시 for문으로 들어가서 조건 참이니까 4출력하고 5출력하고 끝나겠네. (3은 출력 X)

▼ 실무 예시

간단 예시 코드 (로봇 이동 / 배터리)

```
#include <iostream>

int main() {
    int battery = 100;
    bool obstacle_detected = false;

    // 1. 로봇이 최대 10단계 이동한다.
    for(int step=1; step<=10; step++){
        std::cout << "step : " << step << std::endl;

        // 2. 5단계에서 장애물이 감지된다.
        if(step == 5){
            obstacle_detected = true;
        }

        // 3. 장애물이 감지되거나 배터리가 20 미만이면 정지한다.
        if (obstacle_detected || battery < 20) {
            std::cout << "Robot stopped." << std::endl;

            if (obstacle_detected) {
                std::cout << "!!! obstacle detected !!!" << std::endl;
            }
            if (battery < 20) {
                std::cout << "!!! battery is lower than 20% !!!" << std::e
ndl;
            }
            break;
        }
    }
}
```

```

// 4. 각 단계마다 배터리가 10씩 감소한다.
battery -= 10;
std::cout << "BATTERY : [";
for (int j = 0; j < 10; j++) {
    if (j < 10-step) {
        std::cout << "■";
    } else {
        std::cout << " ";
    }
}
std::cout << "]" << battery << "%" << std::endl;

std::cout << "Robot moving forward." << std::endl;

for(int i=1; i<=step; i++){
    std::cout << "    "; }
std::cout << ">>>>" << std::endl;
std::cout << "======(OB)" << std::endl;
}

return 0;
}

```

✓ 출력 결과

```

root@2e14ec01f30d:/workspace# g++ for_test.cpp -o test1 && ./test1
step : 1
BATTERY : [■■■■■■■■■■] 90%
Robot moving forward.
>>>>
======(OB)
step : 2
BATTERY : [■■■■■■■■■] 80%
Robot moving forward.
>>>>
======(OB)
step : 3
BATTERY : [■■■■■■■■] 70%
Robot moving forward.
>>>>
======(OB)
step : 4
BATTERY : [■■■■■■■] 60%
Robot moving forward.
>>>>
======(OB)
step : 5
Robot stopped.
!!! obstacle detected !!!

```

예시 코드 (각 함수 호출 확인 코드)

```

#include <iostream>

void print_robot_start_message() {
    std::cout << "Robot system starting..." << std::endl;
}

void print_sensor_check_message() {
    std::cout << "Checking sensors..." << std::endl;
}

```



```

}

void print_motor_ready_message() {
    std::cout << "Motor controller ready." << std::endl;
}

int main() {
    print_robot_start_message();
    print_sensor_check_message();
    print_motor_ready_message();

    return 0;
}

```

✓ 출력 결과

```

root@a3e5afa32c1b:/workspace# g++ robot_functions.cpp -o function
root@a3e5afa32c1b:/workspace# ./function
Robot system starting...
Checking sensors...
Motor controller ready.

```

🔄 코드 순서 흐름

```

main() 시작
↓
print_robot_start_message() 호출
↓
Robot system starting... 출력
↓
main()으로 복귀
↓
print_sensor_check_message() 호출
↓
Checking sensors... 출력
↓
main()으로 복귀
↓
print_motor_ready_message() 호출
↓
Motor controller ready. 출력
↓
main()으로 복귀
↓
return 0
↓
프로그램 종료

```

▼ 함수 호출

! 선언없이 호출하면 오류

```
#include <iostream>
```

```
int main() {
    print_message();
    return 0;
}

void print_message() {
    std::cout << "Hello" << std::endl;
}
```

위에서부터 아래로 진행하는데 main 함수 돌렸는데 print_message() 함수를 아직 모르니까 에러.

✓ 올바른 함수 선언 방식

```
#include <iostream>

void print_message();    # 미리 이런 함수가 나중에 사용된다고 선언해줌

int main() {
    print_message();
    return 0;
}

void print_message() {
    std::cout << "Hello" << std::endl;
}
```

값 전달 방식

```
#include <iostream>

void increase_battery_fake(int battery) {
    battery = battery + 10;
    std::cout << "Inside function: " << battery << std::endl;
}

int main() {
    int battery = 50;

    increase_battery_fake(battery);

    std::cout << "Inside main: " << battery << std::endl;

    return 0;
}
```

순서

main의 battery 값 50이 함수로 복사됨
 함수 안의 battery는 복사본
 복사본을 60으로 바꿔도 원본은 그대로 50

실무 예시 코드

```

#include <iostream>
#include <string>

// 함수 미리 선언
std::string get_battery_status(int battery);
bool is_obstacle_dangerous(double distance);
double calculate_safe_speed(double target_speed, bool obstacle_dangerous);

// 4. main에서 다음 값으로 테스트한다.
//   - battery = 25
//   - distance = 0.3
//   - target_speed = 1.2
int main() {
    int battery = 25;
    double distance = 0.3;
    double target_speed = 1.2;
    bool obstacle_dangerous = false;

    std::cout << "battery = " << get_battery_status(battery) << std::endl;
    std::cout << "distance = " << is_obstacle_dangerous(distance) << std::endl;
    std::cout << "target_speed = " << calculate_safe_speed(target_speed, obstacle
_dangerous) << std::endl;

    return 0;
}

// 1. get_battery_status(int battery) 함수를 만든다.
std::string get_battery_status(int battery){
    std::string ret = "-";    // return 값 초기화

    bool cond_1 = battery >= 70;
    bool cond_2 = battery >= 30;

    //   - 70 이상이면 "Normal"
    if (cond_1) {
        ret = "Normal";
    }
    //   - 30 이상이면 "Warning"
    else if (cond_2) {
        ret = "Warning";
    }
    //   - 그 외에는 "Low"
    else {
        ret = "LOW";
    }

    return ret;
}

// 2. is_obstacle_dangerous(double distance) 함수를 만든다.
bool is_obstacle_dangerous(double distance){
    bool ret = false;    // return 값 초기화

```

```

// - distance가 0.5 미만이면 true
if (distance < 0.5) {
    ret = true;
}
// - 아니면 false
else {
    ret = false;
}

return ret;
}

// 3. calculate_safe_speed(double target_speed, bool obstacle_dangerous) 함수를 만
든다.
double calculate_safe_speed(double target_speed, bool obstacle_dangerous){
    double ret = 0.0;

    // - obstacle_dangerous가 true이면 0.0 반환
    if (obstacle_dangerous == true) {
        ret = 0.0;
    }

    // - target_speed가 1.5보다 크면 1.5 반환
    else if (target_speed > 1.5) {
        ret = 1.5;
    }

    // - 그 외에는 target_speed 반환
    else {
        ret = target_speed;
    }

    return ret;
}

```

▼ std::vector (배열/리스트)

= python에서의 리스트와 같음.

배열 간단 예시

```

double distances[5] = {1.2, 0.8, 2.5, 0.4, 1.0};

std::cout << distances[3] << std::endl;

```

! 배열의 단점

- 크기가 고정된다.
- 배열의 길이를 직접 관리해야 한다.
- 범위를 벗어난 접근을 조심해야 한다.
- 값을 추가하거나 삭제하기 불편하다.

따라서 배열 대신 크기를 고정하지 않아도 되는 `std::vector` 를 사용한다.

✓ `std::vector`가 필요한 이유

- 감지된 객체 수가 매 프레임 달라짐
- 경로 포인트 개수가 상황마다 달라짐
- 센서 목록이 설정에 따라 달라짐
- 토픽 목록이 늘어남

★ 미리보는 간단 실무 코드

```
struct Robot {
    int id;
    double x, y;
    int battery;
    std::string status;
};

// 실무 핵심 패턴: 구조체를 담은 벡터
std::vector<Robot> robot_fleet;

// 로봇이 추가될 때마다 간단하게 추가
robot_fleet.push_back({1, 10.5, 20.1, 85, "IDLE"});
robot_fleet.push_back({2, 5.2, -1.0, 30, "CHARGING"});

// 모든 로봇을 순회하며 상태 확인 (아까 배운 범위 기반 for문!)
for (const auto& robot : robot_fleet) {
    if (robot.battery < 30) {
        std::cout << "△ 로봇 " << robot.id << "번 충전 필요!" << std::endl;
    }
}
```

! 헛갈리는 코드 보기

```
for (const auto& robot : robot_fleet) {
    // 내용물 사용
}
```

아래 파이썬 for문 보면 이해될 것.

```
for robot in robot_fleet:
```

 std::vector 간단 예시 (1)

```

#include <iostream>
#include <vector>

int main() {
    std::vector<double> distances = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7, 8.8, 9.9,
    10.1};

    std::vector<int> sel_index = {3, 7, 9};
    for (std::size_t i = 0; i < sel_index.size(); i++) {
        std::cout << "data " << sel_index[i] << ": " << distances[sel_index[i] -
    1] << std::endl;
    }

    // 이렇게 해도 됨.
    //for (int i = 0; i < distances.size(); i++) {
    //    if (i == 2 || i == 6 || i == 8) {
    //        std::cout << "data " << i + 1 << " : " << distances[i] << std::endl;
    //    }
    //    else {
    //        continue;
    //    }
    //}

    return 0;
}

```

 출력 결과

```

root@9ba701471c62:/workspace# g++ test3.cpp -o test3
root@9ba701471c62:/workspace# ./test3
data 3: 3.3
data 7: 7.7
data 9: 9.9

```

 std::vector 간단 예시 (2)

```

#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> sensor_names = {
        "front_lidar",
        "rear_lidar",
        "left_camera",
        "right_camera"
    }

    for (const std::string& name : sensor_names) {           // 아래 확인
        std::cout << "Sensor: " << name << std::endl;
    }
}

```

```

    }

    return 0;
}

```

💡 for (const std::string& name : sensor_names) 의미

```

// 이름 하나를 꺼낼 때마다 복사본을 만듦. (메모리 낭비)
for (std::string name : sensor_names)

// 직접 그 주소로 가서 읽음. (복사 안 함, 매우 빠름)
for (std::string& name : sensor_names)

// 직접 가서 읽되, 실수로 내용을 바꾸지는 않겠다는 뜻. (가장 안전하고 빠름)
for (const std::string& name : sensor_names)

```

💡 주의해야 할 것들

📄 예시 1 (x, y 크기 불일치)

```

std::vector<double> path_x = {0.0, 1.0, 2.0};
std::vector<double> path_y = {0.0, 1.0};

```

✅ 해결방법

```

if (path_x.size() != path_y.size()) {    // 각 사이즈 확인할 것.
    std::cout << "Path data error." << std::endl;
}

```

📄 예시 2 (vector 비었는데 출력 시킴)

```

std::vector<double> distances;
double first = distances[0];

```

✅ 해결방법

```

if (!distances.empty()) {    // 비었는지 확인할 것.
    double first = distances[0];
}

```

C++(std::vector) VS Python(리스트)

항목	C++ (std::vector)	Python (list)
선언 방식	<code>std::vector<int> nums {}</code>	<code>nums = []</code>
데이터 추가	<code>nums.push_back(10);</code>	<code>nums.append(10)</code>
특징	한 가지 자료형만 가능	여러 자료형 혼합 가능

`std::vector` 이해해야 할 것들

1. 배열은 같은 자료형의 값을 여러 개 저장한다.
2. 배열 인덱스는 0부터 시작한다.
3. 배열은 크기가 고정되어 있다.
4. `std::vector`는 크기가 변할 수 있는 배열이다.
5. `vector`의 크기는 `.size()`로 확인한다.
6. `vector`에 값 추가는 `.push_back()`을 사용한다.
7. `vector`가 비어 있을 때 `[0]`으로 접근하면 위험하다. (`.empty`로 확인)
8. `std::string`은 문자열을 다룰 때 사용한다.
9. 토픽 이름, 프레임 이름, 로봇 이름은 문자열로 표현할 수 있다.
10. ROS2의 센서 데이터, 경로 포인트, 메시지 목록은 `vector` 개념과 연결된다.

▼ 포인터

💡 왜 포인터를 배울까?

YOLO로 작업할 때 동영상을 계속 영상 전체를 복사해서 넘겨주면 메모리랑 작업량 너무 많음.

따라서 그 영상 ID(주소)만 들고 다니다가 필요할 때 ID(주소) 넘겨주면 됨.

포인터도 마찬가지다.

그냥 필요할 때 주소에 가서 찾으면서 효율적으로 사용하기 위해 포인터를 사용한다.

💬 포인터 간단 설명

포인터는 변수다.

C++은 변수를 만들면 컴퓨터 메모리 어딘가에 값을 저장한다.

```
int battery = 85;
```

↓ 저장된 형태

메모리 주소	값
0x7ffc1234abcd	85

주소를 확인할 때는 `&`를 변수 앞에 붙이면 된다.

```
&battery
```

📄 간단 예시 코드

```
#include <iostream>

int main() {
    int battery = 85;

    std::cout << "Battery value: " << battery << std::endl;
    std::cout << "Battery address: " << &battery << std::endl;
```



```
    return 0;
}
```

✓ 출력 결과

```
root@3ba701471c62:/workspace# g++ test.cpp -o test
root@3ba701471c62:/workspace# ./test
Battery value: 85
Battery address: 0x7ffdf1046a84
```

포인터 / 참조

구분	포인터	참조
의미	주소를 저장하는 변수	기존 변수의 별명
선언 예	<code>int* p = &x;</code>	<code>int& r = x;</code>
값 접근	<code>*p</code>	<code>r</code>

포인터 간단 예시 (1)

```
#include <iostream>

void update_battery_by_pointer(int* battery_ptr) { // 주소를 받으려고 포인터로
    *battery_ptr = 50; // 값을 저장하기 위해 * 앞에 표시
}

int main() {
    int battery = 85;

    std::cout << "Origin Battery : " << battery << std::endl;

    update_battery_by_pointer(&battery); // battery 변수의 주소를 넣음.

    std::cout << "Changed Battery : " << battery << std::endl;

    return 0;
}
```

👍 코드 쉽게 이해하기

```
int battery = 85;
int* battery_ptr = &battery; // battery의 주소를 battery_ptr에 넣음.

std::cout << battery_ptr; // 결과: 0x1234 (주소값이 나옴)
std::cout << *battery_ptr; // 결과: 85 (주소를 따라가서 그 안의 값을 꺼냄)
```

✓ 출력 결과

```
root@3ba701471c62:/workspace# g++ test.cpp -o test
root@3ba701471c62:/workspace# ./test
Origin Battery : 85
Changed Battery : 50
```

포인터 간단 예시 (2)

```

#include <iostream>
#include <string>

int main() {
    std::string robot_name = "turtlebot3";
    int battery = 80;
    double speed = 0.5;

    std::string* name_ptr = &robot_name;
    int* battery_ptr = &battery;
    double* speed_ptr = &speed;

    std::cout << "Robot name: " << *name_ptr << std::endl;
    std::cout << "Battery: " << *battery_ptr << "%" << std::endl;
    std::cout << "Speed: " << *speed_ptr << " m/s" << std::endl;

    *battery_ptr = 60;
    *speed_ptr = 1.0;

    std::cout << "==== After update through pointers =====> << std::endl;
    std::cout << "Battery: " << battery << "%" << std::endl;
    std::cout << "Speed: " << speed << " m/s" << std::endl;

    return 0;
}

```

✓ 출력 결과

```

root@3ba701471c62:/workspace# g++ test.cpp -o test
root@3ba701471c62:/workspace# ./test
Robot name: turtlebot3
Battery: 80%
Speed: 0.5 m/s
==== After update through pointers =====
Battery: 60%
Speed: 1 m/s

```

포인터에서 > 연산자 이해하기

ROS2에서 자주 보이는 연산자임.

```
msg->data
```

포인터가 가리키는 객체의 멤버에 접근하는 문법이다.

먼저 객체를 알아보면,

```

struct RobotStatus {
    int battery;
    double speed;
};

```

일반 객체에서는 점 . 을 사용한다.

```

RobotStatus status;
status.battery = 80;

```

```
status.speed = 0.5;
```

포인터에서는 화살표 `->` 를 사용한다.

```
RobotStatus* status_ptr = &status;
status_ptr->battery = 60;
status_ptr->speed = 1.0;
```

`status_ptr->battery` 는 다음과 같은 의미이다.

```
(*status_ptr).battery
```

▼ Python / C++ 대표적인 문법 차이 모음

항목	C++ (규칙과 성능)	Python (간결과 편의)	한 줄 요약
시작점	<code>int main() { }</code>	(필요 없음)	C++은 입구가 정해져 있음
입력	<code>std::cin >> x;</code>	<code>x = input()</code>	받는 방향(<code>>></code>) vs 넣는 방향(<code>=</code>)
출력	<code>std::cout << x;</code>	<code>print(x)</code>	보내기(<code><<</code>) vs 출력하기(<code>print</code>)
마침표	<code>;</code> (세미콜론)	(없음)	C++은 문장 끝에 반드시 <code>;</code>
영역	<code>{ }</code> (중괄호)	들여쓰기	C++은 가두고, Python은 민다
변수	<code>int x = 10;</code>	<code>x = 10</code>	C++은 이름표(타입)가 필수
논리	<code>true / false</code>	<code>True / False</code>	Python은 앞글자가 대문자
상수	<code>const int X = 1;</code>	<code>X = 1</code>	Python은 관례로만 '상수' 취급
함수	<code>void f() { }</code>	<code>def f():</code>	반환타입(<code>void</code>) vs 정의(<code>def</code>)
조건	<code>if (x > 0) { }</code>	<code>if x > 0:</code>	Python은 괄호 대신 콜론(<code>:</code>)
반복	<code>for(i=0; i<5; i++)</code>	<code>for i in range(5):</code>	Python은 범위(<code>range</code>) 중심
배열	<code>int a[3];</code>	<code>a = []</code>	C++은 크기가 고정됨
비어있음	<code>nullptr</code>	<code>None</code>	값이 없음을 나타내는 단어 차이
연결	<code>&&, , !</code>	<code>and, or, not</code>	C++은 기호, Python은 영어 단어
나눗셈	<code>/</code> (정수/실수 구분)	<code>/</code> (실수), <code>//</code> (정수)	Python은 정수 몫 기호가 따로 있음
주석	<code>//</code> 한 줄, <code>/*</code> 여러줄 <code>*/</code>	<code>#</code> 한 줄, <code>"""</code> 여러줄 <code>"""</code>	메모(주석)를 남기는 기호 차이
가져오기	<code>#include <.></code>	<code>import ..</code>	라이브러리를 불러오는 방식
클래스	<code>class A { };</code>	<code>class A:</code>	Python은 뒤에 콜론(<code>:</code>) 필수
접근제한	<code>private:</code> , <code>public:</code>	<code>_variable</code> (관례)	C++은 강제, Python은 약속
메모리	직접 관리 (<code>delete</code>)	자동 관리 (GC)	C++은 청소를 직접, Python은 자동